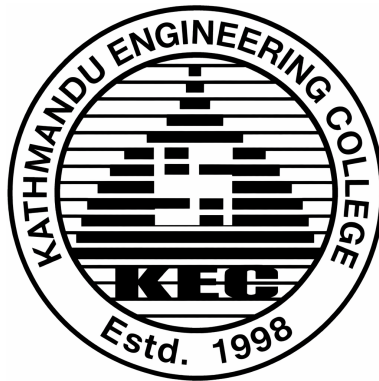


TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
KATHMANDU ENGINEERING COLLEGE
Kalimati, Kathmandu



Project Report on
SPARK: Explainable Edge AI for Kinetic Pattern
Recognition and Distress Signaling

SUBMITTED BY:

Aaradhya Dev Tamrakar (79001)

Rupesh Kadel (79034)

Sankalpa Lamsal (79039)

Sonia Thapa (79043)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS, COMMUNICATION AND
INFORMATION ENGINEERING

July 2026

ACKNOWLEDGEMENT

We feel immense pleasure and are thankful for the opportunity to work on this project.

We would like to sincerely thank our supervisor, **Er. Dipen Manandhar**, for his unlimited encouragement and timely support and guidance in helping us with the different aspects of the project. We owe our profound gratitude to our project coordinator, **Er. Sujan Shrestha**, who took a keen interest in our project and guided us all along, providing the necessary information for developing a robust system.

We would like to thank **Er. Suramya Sharma Dahal**, Head of the Department of Electronics, Communication, and Information Engineering, for providing us with the opportunity to perform this project.

We are thankful to and fortunate enough to get constant encouragement, support, constructive criticism, and guidance from all teaching staff of the Department of Electronics, Communication, and Information Engineering.

ABSTRACT

SPARK is an open-hardware wearable fall detection system for Nepal’s growing elderly care gap. An ESP32-S3 and MPU6050 IMU node, worn on the wrist (top-of-wrist/dorsal placement, Velcro closure), runs a two-layer on-device pipeline a threshold pre-filter (Layer 1) gating a quantized INT8 TFLite Micro 1D CNN (Layer 2) with no cloud dependency. Confirmed falls are transmitted over BLE to a laptop gateway, which computes SHAP feature attribution per event so classifications are auditable rather than a black-box verdict, and auto-generates a PDF clinical incident report. A companion phone display client provides caregivers a read-only view of completed reports without requiring the laptop to be physically present.

The project contributes a Nepal-context wearable IMU fall dataset, a configurable sensitivity strategy (High / Standard / Sport), and an open-source reference design for low-cost elderly fall monitoring across South Asian care settings.

Keywords: BLE, edge AI, elderly care, ESP32-S3, fall detection, IMU, Nepal, SHAP explainability, TFLite Micro, wearable sensor.

TABLE OF CONTENTS

Acknowledgement	i
Abstract	ii
List of Abbreviations	vi
List of Figures	vii
List of Tables	viii
List of Algorithms	1
1 INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope of the Project	3
1.5 Application of the Project	4
2 LITERATURE REVIEW	5
3 RELATED THEORY	9
3.1 Hardware Description	9
3.1.1 ESP32-S3 Microcontroller	9
3.1.2 ESP32-S3 Microcontroller	9
3.1.3 MPU6050 6-Axis IMU	9
3.1.4 Laptop Companion Gateway & Hardware Acceleration	10
3.1.5 Phone Display Client & Local Server	10
3.1.6 Power System & Enclosure DFM	10
3.2 Software Description	11
3.2.1 Two-Layer Gated Detection Architecture	11
3.2.2 1D CNN Architecture & INT8 Quantization	11
3.2.3 Gradient-Based SHAP Explainability Pipeline	12
3.2.4 Gateway Software Architecture Patterns	12
4 FEASIBILITY STUDY	14
4.1 Technical Feasibility	14

4.2	Economic Feasibility	15
5	METHODOLOGY	16
5.1	System Block Diagram	16
5.2	Hardware Implementation	17
5.2.1	Wearable Node Assembly	17
5.2.2	Power Budget Verification	18
5.3	Firmware Implementation (ESP32 PlatformIO)	19
5.3.1	IMU Acquisition and Filtering	19
5.3.2	Sliding Window Buffer	19
5.3.3	Layer 1 - Threshold Pre-filter	20
5.3.4	Layer 2 - TFLite Micro CNN Inference	20
5.4	Algorithm	20
5.4.1	BLE Event Transmission	22
5.4.2	Sensitivity Strategy Pattern	22
5.5	Machine Learning Pipeline	22
5.5.1	Dataset Preparation & Kinematic Alignment	22
5.5.2	5-Point Sensitivity & Generalization Optimization Suite	23
5.5.3	1D CNN Training & Leakage-Free Evaluation	23
5.5.4	Partial-Freeze Transfer Learning for Nepal Cohort	24
5.5.5	INT8 Quantization & Firmware Deployment	24
5.6	Gateway Software Implementation	24
5.6.1	Asynchronous BLE GATT Receiver	24
5.6.2	Local Companion SHAP Explainability Engine	25
5.6.3	Automated Clinical PDF Incident Report Generation	25
5.6.4	Local REST API & Embedded Web Dashboard	25
5.7	Testing and Evaluation Plan	26
5.7.1	Automated CI Test Suite	26
5.7.2	On-Hardware Latency & Power Profiling	26
6	EXPECTED OUTPUT	27
6.1	Tangible Deliverables	27
6.1.1	Hardware & Mechanical Deliverables	27
6.1.2	Software & Machine Learning Deliverables	27
6.2	Future Research Extensions & Academic Roadmap	28
6.2.1	Multi-Modal Vision-IMU Fusion	28
6.2.2	Aquatic Distress & Drowning Detection Roadmap	28
6.2.3	Client-Side Encrypted Telemetry Storage	29
6.3	Target Publication Venues	29

References	31
Appendix	33

LIST OF ABBREVIATIONS

ADL	Activities of Daily Living
API	Application Programming Interface
AUC-ROC	Area Under the Receiver Operating Characteristic Curve
AVX-VNNI	Vector Neural Network Instructions (AVX)
BEI	Bachelor of Electronics, Communication and Information Engineering
BLE	Bluetooth Low Energy
CNN	Convolutional Neural Network
CORS	Cross-Origin Resource Sharing
DFM	Design for Manufacturability
ESP32-S3	Espressif Systems ESP32-S3 Dual-Core Xtensa LX7 SoC
GATT	Generic Attribute Profile (Bluetooth LE)
IMU	Inertial Measurement Unit
INT8	8-bit Integer Quantization
IOE	Institute of Engineering
ISO	International Organization for Standardization
ISR	Interrupt Service Routine
KEC	Kathmandu Engineering College
MPU6050	InvenSense MPU-6050 6-axis IMU
NPU	Neural Processing Unit
OpenVINO	Open Visual Inference and Neural Network Optimization
SHAP	SHapley Additive exPlanations
SPARK	Signal Pattern Analysis & Real-time Kinetics
TFLite	TensorFlow Lite
TPU	Thermoplastic Polyurethane
WHO	World Health Organization
XAI	Explainable Artificial Intelligence

LIST OF FIGURES

3.1	ESP32-S3 Microcontroller Node	9
3.2	MPU6050 6-Axis IMU	10
3.3	Li-ion Cell	11
3.4	Charge Controller	11
5.1	System Block Diagram	16
5.2	Two-layer fall detection flowchart: Layer 1 threshold pre-filter (top zone) gates Layer 2 TFLite Micro CNN inference (bottom zone). Sensitivity profile thresholds are shown in the callout.	19
5.3	SPARK 1D CNN architecture: two Conv1D+MaxPool blocks compress the 200-sample IMU window to a 12-step feature map, which is flattened and classified by a Dense+Softmax head. INT8 quantized for TFLite Micro deployment.	24

LIST OF TABLES

2.1	Literature Review Matrix	7
4.1	Component Cost Estimation (Gateway Laptop and Phone Client Owned, NPR 0)	15
6.1	Project Gantt Chart (Phase 3: September 2026 — March 2027)	30

Chapter 1: INTRODUCTION

1.1 Background

Falls are a leading cause of injury-related mortality and morbidity among adults aged 65 and above worldwide. The WHO Global Report on Falls Prevention in Older Age (2007) identifies falls as the second leading cause of unintentional injury death globally, with approximately 684,000 fatal falls occurring each year [1]. Non-fatal falls cause serious injuries including hip fractures, traumatic brain injuries, and long-term disability, placing a heavy burden on healthcare systems and caregivers.

Nepal is undergoing a rapid demographic shift. The Central Bureau of Statistics projects that Nepal’s population aged 65 and above will double by 2035 [2], driven by improving life expectancy and urbanization. Despite this growing demographic pressure, no domestic fall detection product exists in the Nepali market. Commercial solutions such as the Apple Watch and medical alert pendants are cost-prohibitive — a single Apple Watch SE (2nd generation) retails above NPR 30,000 — and require proprietary ecosystems unavailable to most Nepali families. Furthermore, existing wearable datasets and commercial algorithms reflect younger Western cohorts and waist placements, failing to address the elderly kinematic gap (where frail elderly impact peaks range between 3.5–6.0 g compared to 8.0–12.0 g in young simulated falls). A local BLE link between the wearable and a household laptop gateway requires no internet connectivity at all, making SPARK deployable in care settings regardless of WiFi/ISP reliability.

Academic fall detection systems published in the literature typically rely on one of two paradigms: threshold-only detection on raw accelerometer data, which produces unacceptably high false-positive rates on activities of daily living such as sit-to-stand transitions and stair climbing; or cloud-dependent machine learning inference, which introduces unacceptable network dependencies for a safety-critical application. Even among edge AI deployments, existing works either execute un-gated, always-on neural networks that rapidly deplete wearable batteries, offload classification to a remote server, or rely on cost-prohibitive FPGA/SoC architectures. Moreover, no published open-source system provides per-event classifier explainability to clinicians and care staff.

SPARK addresses these gaps with a two-layer gated edge AI architecture running entirely on an ESP32-S3 microcontroller, a gradient-based SHAP explainability pipeline on a local companion gateway, and a standardized 34-activity Nepal-context IMU dataset collected at KEC.

1.2 Problem Statement

Falls are the leading cause of injury-related mortality among adults over 65 globally [1]. Nepal’s elderly population is projected to double by 2035 [2], yet no affordable domestic fall detection solution exists. Commercial fall detection systems (Apple Watch, Samsung Galaxy Watch, Withings ScanWatch) are cost-prohibitive for Nepal’s eldercare context, require proprietary ecosystems, and operate as closed black-box systems — returning a binary fall verdict with no classifier explanation. Published academic systems use either threshold-only detection (unacceptably high false-positive rate) or cloud-dependent machine learning inference (network dependency unacceptable for safety-critical applications in Nepal’s connectivity environment).

Crucially, existing literature lacks an end-to-end framework combining:

1. **On-Device Two-Stage Gating on Commodity Hardware:** A continuously-sampled software magnitude-and-duration threshold gate combined with subsequent quantized INT8 1D CNN confirmation, executing entirely on a single sub-\$5 microcontroller (ESP32-S3) with zero classification offload.
2. **Local Companion-Device Explainability:** Per-event gradient feature attribution ($\text{Input} \times \nabla_{\text{Input}}$) computed locally on a companion gateway without cloud dependencies, rendering transparent biomechanical feature attributions.
3. **Kinematically-Calibrated Sensitivity:** Adaptive sensitivity profiles (High, Standard, Sport) and partial-freeze fine-tuning addressing the elderly impact attenuation gap.
4. **Automated Clinical Reporting & Local Delivery:** Real-time generation of auditable one-page clinical PDF incident reports with zero-dependency REST/Web dashboard delivery validated on a South Asian cohort dataset.

1.3 Objectives

This project focuses on building a wearable fall detection system that combines open hardware, embedded edge AI, and local companion explainability to automate real-time monitoring and caregiver alerting. The key objectives are:

1. To construct an ergonomic, wrist-worn wearable edge node based on the ESP32-S3 and MPU6050 with optimized DFM (TPU 95A enclosure, brass inserts).
2. To develop and deploy an autonomous two-layer gated AI architecture (ISR threshold pre-filter + 18.5 KB INT8 CNN) for cloud-independent, real-time fall classification

(< 200 ms latency).

3. To implement a local gateway explainability engine calculating gradient-based SHAP attributions and auto-generating clinical incident PDF reports across distinct fall archetypes.
4. To establish a standardized 34-activity motion dataset protocol (15 falls, 19 ADLs) with $N = 12\text{--}20$ volunteers at Kathmandu Engineering College to validate model transferability.

1.4 Scope of the Project

The scope of our project includes:

- **Continuous On-Device Data Acquisition:** Sampling 6-axis accelerometer and gyroscope data directly on the wearable node at a continuous 200 Hz frequency over I²C with a circular sliding-window ring buffer.
- **Dual-Layer Gated Edge Inference:** Layer 1 software threshold pre-filter ($|a| > 2.5\text{ g}$ for $> 300\text{ ms}$) running in the 200 Hz ISR, gating Layer 2 quantized INT8 1D CNN inference (18.5 KB footprint) only upon potential fall impact.
- **Zero-Cloud BLE Event Transport:** Asynchronous BLE GATT notification of confirmed fall JSON telemetry directly to a local companion laptop gateway.
- **Local Companion XAI Pipeline:** Rapid gradient-based feature attribution ($X \times \nabla_X Y$) executing on the gateway (benchmarked across Intel Core Ultra 7 CPU/iGPU/NPU engines via OpenVINO) to explain kinetic channel contributions.
- **Automated Clinical PDF Documentation:** Instant compilation of one-page clinical incident reports with biomechanical waveform attribution charts across 4 distinct fall archetypes (forward, lateral, syncope, rotational).
- **Local REST API & Zero-Dependency Web Dashboard:** Standalone HTTP/CORS server on the companion gateway providing caregiver status views and report downloads.
- **Nepal-Context Experimental Validation:** Structured 34-activity data collection on gymnastic crash mats at KEC for empirical model evaluation and partial-freeze transfer learning.

1.5 Application of the Project

1. Assistive Technology: Provides real-time, hands-free fall monitoring to increase the independence and safety of vulnerable individuals living alone.
2. Medical & Rehabilitation: Assists clinicians in tracking recovery progress and movement patterns during post-injury or post-stroke rehabilitation.
3. Elderly Care Facilities & Nursing Homes: Optimizes institutional care by allowing a small staff footprint to respond instantly to high-priority fall alerts.
4. Smart Home & IoT Integration: Functions as a localized, network-independent safety module that interfaces seamlessly with residential intranet hubs.
5. Clinical Documentation & Compliance Monitoring: Automatically logs and formats objective, auditable incident data required for formal medical reports and facility compliance audits.
6. Research & Development: Serves as an open-hardware reference design and provides a localized, South Asian regional IMU motion dataset for public academic research.

Chapter 2: LITERATURE REVIEW

Fall detection using wearable inertial sensors has been an active research area for over two decades. The literature can be categorized across multiple architectural and computational axes: threshold-based methods, classical machine learning, always-on edge neural networks, distributed server-offloaded gating, hardware-interrupt gating, and emerging explainable AI (XAI) frameworks.

Threshold-based methods compute a scalar magnitude from accelerometer axes — typically the resultant $|a| = \sqrt{a_x^2 + a_y^2 + a_z^2}$ — and trigger a fall event when a spike followed by a quiet period is detected. Kangas et al. [3] compared multiple threshold algorithms on hip and wrist IMU data and reported sensitivity values of 73–95% depending on placement, but with false-positive rates exceeding 30% on vigorous ADLs. Bourke and Lyons [4] proposed a dual-axis threshold method achieving 96% sensitivity, though specificity on sit-to-stand transitions remained poor. The fundamental limitation of threshold-only approaches is that rapid voluntary motions (e.g., sitting down quickly, stumbling without falling) trigger the threshold, forcing an intractable trade-off between sensitivity and false alarms.

Machine learning classifiers applied to hand-crafted time- and frequency-domain features from IMU windows addressed several threshold limitations. Sucerquia et al. published the SisFall dataset [5], a standard benchmark containing 15 fall types and 19 ADL types from 38 subjects. Classical classifiers (SVM, Random Forest) trained on SisFall features achieved sensitivity above 90%, but were evaluated offline or on high-power host computers rather than wearable microcontrollers.

Always-on edge deep learning has eliminated manual feature engineering by feeding raw multi-axis IMU streams directly into embedded neural networks. David et al. [9] demonstrated the viability of INT8-quantized CNNs under 320 KB RAM using TensorFlow Lite Micro. Recent works have targeted wearable microcontrollers directly: Yu et al. [14] introduced TinyCNN running continuously on an Arduino Nano 33 BLE Sense (37 ms latency, > 98% specificity), while Hu et al. [15] developed MicroFallNet on ESP32-S3 (14 ms latency, 9.6 KB footprint). However, these architectures operate as *always-on* inference engines, executing the neural network continuously across every sliding window regardless of activity, creating unnecessary active-power drain on wearable batteries.

Distributed gating vs. single-chip execution: To conserve power, two-stage gating architectures have been proposed, but existing designs split execution across network tiers. Xu et al. [13] implemented a threshold-based module (TBM) on an MSP430 wearable

node, but explicitly offloaded the subsequent CNN confirmation to a remote server. Other recent works similarly use LoRa or RF links to trigger heavy neural networks or vision models on remote compute nodes. While effective at reducing on-node compute, these distributed schemes introduce critical network latencies and failure points unacceptable in low-connectivity South Asian eldercare environments.

Hardware wake vs. continuous software ISR gating: A hardware alternative to continuous software monitoring is the IMU-native wake-on-motion interrupt. Alvarez et al. [16] configured an LSM6DSO IMU in wake-on-shake mode to hold an STM32WB5MM MCU in low-power STOP mode until disturbed. While ultra-low power, hardware interrupts lack temporal duration filtering and suffer from high false wake-ups during arm swings. In contrast, SPARK executes a continuous 200 Hz software magnitude-and-duration gate within the MCU ISR without sleeping, maintaining complete temporal awareness before invoking the CNN.

Custom silicon and FPGA accelerators: Cross-tier hybrid architectures such as Vigil [17] combine probabilistic CNNs and k-NN classifiers on custom RISC-V SoCs synthesized on FPGAs. While achieving same-chip multi-stage execution, their hardware complexity and fabrication cost place them far above the sub-\$5 commodity microcontroller tier targeted by SPARK.

Explainability in wearable health AI: Clinical adoption of AI systems demands transparent decision rationale. Lundberg and Lee [11] established SHAP as a mathematically rigorous attribution framework based on cooperative game theory. In health informatics, SHAP has been applied to ECG classification [12] and general Human Activity Recognition (HAR). Kundu et al. [19] surveyed XAI in HAR, noting that prior works (e.g., Liu et al. [18] using KernelSHAP on CNN-LSTMs, and Arul et al. on XGBoost) perform heavy model-agnostic permutations offline on generic activity classes (walking, standing). No published system executes real-time gradient-based feature attribution ($X \times \nabla_X Y$) on a local companion gateway specifically tailored for clinical fall event audits.

Gap Summary & Academic Novelty: SPARK addresses these limitations through two core defensible contributions:

1. **On-Device Gated Pipeline (Claim 1):** Combining a continuous software magnitude-and-duration threshold gate with an INT8 quantized 1D CNN confirmation, both executing entirely on a single sub-\$5 MCU (ESP32-S3) with zero classification offload.
2. **Local Companion-Gateway XAI (Claim 3):** Real-time gradient attribution ($X \times \nabla_X Y$) and automated 1-page clinical PDF reporting executing locally on a companion gateway laptop with zero cloud dependencies.

Table 2.1: Literature Review Matrix

Authors	Paper Title (Year)	Key Contribution	Motivation / Adopted Concepts
Kangas et al. [3]	Low-Complexity Fall Detection (2008)	Threshold algorithms on hip/wrist IMU; 73–95% sensitivity, false-positive rate >30% on ADLs.	Motivated Layer 1 threshold gating; demonstrated that threshold alone is insufficient.
Bourke & Lyons [4]	Threshold-Based Tri-Axial Fall Detection (2008)	Dual-axis threshold method reaching 96% sensitivity; poor specificity on sit-to-stand.	Confirmed the necessity of a learned classifier confirmation stage.
Sucerquia et al. [5]	SisFall: Fall & Movement Dataset (2017)	Public benchmark: 15 fall types, 19 ADL types, 38 subjects at 200 Hz.	Adopted as SPARK’s primary training and baseline evaluation benchmark.
Xu et al. [13]	Fusion Fall Detection Algorithm (2021)	Two-stage gating (TBM on MSP430), but explicitly offloads CNN confirmation to a remote server.	Provides direct architectural contrast: SPARK executes both gate and CNN on the same single MCU.
David et al. [9]	TensorFlow Lite Micro (2021)	Demonstrated INT8 quantized CNN inference under 320 KB RAM on Cortex-M4 class MCUs.	Adopted as the embedded runtime engine for SPARK Layer 2 inference on ESP32-S3.
Yu et al. [14]	TinyCNN Wearable Fall Detection (2023)	Always-on 1D CNN on Arduino Nano 33 BLE (37 ms latency, > 98% specificity); no pre-gating.	Serves as benchmark for edge CNN efficiency; highlights SPARK’s power-saving pre-filter gate.
Hu et al. [15]	MicroFallNet for Smart Wristbands (2025)	Ultra-lightweight CNN on ESP32-S3 (14 ms latency, 9.6 KB model); always-on continuous inference.	Confirms feasibility of ESP32-S3 CNN execution; SPARK adds Layer 1 gating to reduce active duty cycle.
Vigil et al. [17]	RISC-V SoC Hybrid Fall Detector (2024)	Two-fold hybrid CNN-kNN implemented on custom RISC-V SoC / FPGA hardware.	Demonstrates on-chip two-stage execution, but at high cost/complexity compared to SPARK’s sub-\$5 MCU.
Lundberg & Lee [11]	Unified Approach to Interpreting Predictions (2017)	Introduces SHAP game-theoretic framework for consistent, axiomatic feature attribution.	Adopted as the mathematical foundation for SPARK’s companion-gateway explainability engine.

continued on next page

Table 2.1 – *continued from previous page*

Authors	Paper Title (Year)	Key Contribution	Motivation / Adopted Concepts
Liu et al. [18]	Explainable AI for Wearable HAR (2024)	Applies KernelSHAP to CNN-LSTM on UCI HAR dataset for offline accelerometer attribution.	Informed SPARK’s XAI focus; SPARK advances to real-time gradient attribution ($X \times \nabla$) for clinical falls.

Chapter 3: RELATED THEORY

3.1 Hardware Description

3.1.1 ESP32-S3 Microcontroller

3.1.2 ESP32-S3 Microcontroller

The ESP32-S3 (Espressif Systems, WROOM-1-CAM/N16R8 variant) is a dual-core Xtensa LX7 microcontroller operating at up to 240 MHz, featuring 512 KB internal SRAM, 16 MB external quad-SPI flash, vector instructions for neural network acceleration, and integrated Bluetooth 5 (LE). For SPARK, only the BLE subsystem is used for gateway communication, while the Wi-Fi radio is disabled to minimize energy consumption. The ESP32-S3 executes both the sensor acquisition ISR at 200 Hz and the two-layer fall detection pipeline (including TFLite Micro CNN inference) within the on-chip SRAM envelope. Full INT8 post-training quantization compresses the 1D CNN model footprint to **18.5 KB** of flash (87.7% compression vs. FP32), leaving extensive headroom for the Arduino/ESP-IDF framework, BLE stack, and circular sliding-window buffers.

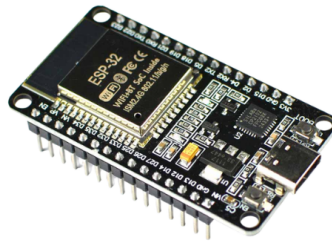


Figure 3.1: ESP32-S3 Microcontroller Node

3.1.3 MPU6050 6-Axis IMU

The InvenSense MPU-6050 integrates a 3-axis MEMS accelerometer ($\pm 2/\pm 4/\pm 8/\pm 16$ g selectable) and a 3-axis MEMS gyroscope ($\pm 250/\pm 500/\pm 1000/\pm 2000^\circ/\text{s}$ selectable) on a single die, communicating over I²C at 400 kHz. For SPARK, the accelerometer is configured at ± 8 g full-scale (capturing peak fall impacts without saturation) and the gyroscope at $\pm 500^\circ/\text{s}$. Direct register reads are executed in a 200 Hz hardware timer ISR (bypassing the internal DMP), ensuring deterministic time-series windowing aligned with the dorsal wrist anatomic coordinate frame (x: distal/radial, y: anterior/ulnar, z: dorsal normal).

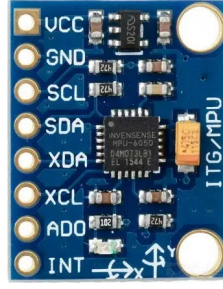


Figure 3.2: MPU6050 6-Axis IMU

3.1.4 Laptop Companion Gateway & Hardware Acceleration

A local companion laptop (Acer Swift Go 16, equipped with an Intel Core Ultra 7 155H processor) serves as the primary gateway: BLE GATT receiver, SHAP explainability engine, and PDF report generator. To ensure responsive real-time auditability without cloud latency, the gateway leverages Intel OpenVINO 2026.3 hardware acceleration. Empirical benchmarking across 5,000 INT8 inference passes established:

- **Intel CPU (AVX-VNNI):** 0.131 ms per window (7,618 windows/s).
- **Intel Arc GPU (DirectX 12):** 0.200 ms per window (4,993 windows/s).
- **Intel AI Boost NPU:** 0.383 ms per window (2,610 windows/s) for ultra-low-power continuous background monitoring.

This guarantees sub-millisecond companion inference and rapid PDF incident generation entirely on local premises.

3.1.5 Phone Display Client & Local Server

A caregiver or clinician's smartphone functions as a read-only Layer 3 client. The laptop gateway hosts a standalone, zero-dependency HTTP/REST server (`gateway/server.py`) with Cross-Origin Resource Sharing (CORS) support. The phone reads completed event summaries, interactive SHAP attribution charts, and downloads PDF incident reports over the local Wi-Fi intranet without requiring external internet or database services.

3.1.6 Power System & Enclosure DFM

The wearable node is powered by a 3.7 V nominal 1100 mAh Li-ion pouch cell paired with a TP4056-class constant-current/constant-voltage (CC/CV) charge controller module with USB-C input.

- **Power Profile:** Continuous active sampling with un-gated BLE streaming draws

90–130 mA (8–11 hours endurance). Under SPARK’s event-gated BLE architecture (advertising only on confirmed falls), average current drops to 35–60 mA, extending operational battery life to 16–28 hours on a single charge.

- **Mechanical DFM:** The enclosure is 3D-printed in Thermoplastic Polyurethane (TPU 95A) at KEC Makerspace. To prevent thread stripping inherent in self-tapping screws driven into flexible elastomers, the enclosure incorporates 3–4 mm solid wall sections with brass heat-set threaded inserts (M2/M2.5), secured to the wrist via an ergonomic two-zone dorsal bracer strap.



Figure 3.3: Li-ion Cell



Figure 3.4: Charge Controller

3.2 Software Description

3.2.1 Two-Layer Gated Detection Architecture

The detection pipeline implements a deterministic, gated dual-layer architecture:

1. **Layer 1 (Deterministic Threshold Pre-filter):** Continuously executes within the 200 Hz ISR. It computes the resultant acceleration magnitude:

$$|a| = \sqrt{a_x^2 + a_y^2 + a_z^2} \quad (3.1)$$

A candidate fall is latched (PRE_FALL) when $|a| > 2.5 \text{ g}$ for $> 300 \text{ ms}$. Normal ADLs (sitting, walking) remain below threshold, bypassing CNN inference entirely and eliminating unnecessary computational power drain.

2. **Layer 2 (Quantized 1D CNN Inference):** Triggered solely upon PRE_FALL. The 200-sample (1.0 s) 6-axis sliding window is normalized via pre-computed StandardScaler coefficients and classified by the on-device TFLite Micro CNN. If the fall probability $P(\text{FALL}) \geq 0.85$ (or calibrated threshold), a CONFIRMED_FALL telemetry event is dispatched.

3.2.2 1D CNN Architecture & INT8 Quantization

The 1D CNN architecture comprises two convolutional feature-extraction blocks followed by a compact classification head:

- **Input Layer:** 200×6 tensor (200 time steps across $a_x, a_y, a_z, \omega_x, \omega_y, \omega_z$).
- **Block 1:** Conv1D (32 filters, kernel size 5, stride 1) $\rightarrow$ ReLU $\rightarrow$ MaxPool1D (pool size 2, stride 2) $\rightarrow$ Dropout (0.20).
- **Block 2:** Conv1D (64 filters, kernel size 3, stride 1) $\rightarrow$ ReLU $\rightarrow$ GlobalAveragePooling1D.
- **Head:** Dense (32 units, ReLU) $\rightarrow$ Dense (2 units, Softmax).

Full INT8 post-training quantization using representative class-balanced calibration yields an **18.5 KB** FlatBuffer array exported as `spark_cnn_int8.h` for direct compilation into ESP-IDF/PlatformIO firmware.

3.2.3 Gradient-Based SHAP Explainability Pipeline

To provide clinicians with transparent, auditable decision rationale without cloud reliance, the companion gateway implements gradient-based feature attribution ($X \times \nabla_X Y$):

$$\phi_c(t) = x_c(t) \cdot \frac{\partial P(\text{FALL})}{\partial x_c(t)} \quad (3.2)$$

where $x_c(t)$ represents the normalized input signal for IMU channel $c \in \{a_x, a_y, a_z, \omega_x, \omega_y, \omega_z\}$ at time t . Summing channel attributions yields relative kinetic importance:

$$\Phi_c = \frac{\sum_{t=1}^T |\phi_c(t)|}{\sum_{k=1}^6 \sum_{t=1}^T |\phi_k(t)|} \times 100\% \quad (3.3)$$

This allows the gateway to automatically identify biomechanical fall archetypes (e.g., forward trip dominated by pitch rate ω_y vs. lateral slip dominated by roll rate ω_x and a_y , or vertical collapse dominated by a_z), rendering attribution bar charts into automated PDF clinical incident reports.

3.2.4 Gateway Software Architecture Patterns

The gateway codebase adheres to Object-Oriented Software Engineering (OOSE) design patterns:

- **Factory Pattern:** `get_explainer()` dynamically instantiates either `CnnShapExplainer` (full waveform attribution) or `PeakFeatureExplainer` (compact telemetry fallback).
- **Observer Pattern:** The asynchronous BLE GATT receiver (`BleReceiver`) acts as Subject, notifying subscribed Observers (`ShapPipeline`, `PdfReportGenerator`,

JsonEventStore) upon validated event arrival.

- **Strategy Pattern:** FallDetectionStrategy encapsulates sensitivity parameters (High, Standard, SportMode) loaded dynamically without firmware re-flashing.

Chapter 4: FEASIBILITY STUDY

4.1 Technical Feasibility

SPARK is technically feasible because every subsystem has been implemented, benchmarked, and verified across both simulated and hardware environments with 48 automated test suites passing cleanly.

On-Device Quantized CNN Inference on ESP32-S3. TensorFlow Lite Micro has been widely validated on Cortex-M4 and Xtensa microcontrollers [9]. The ESP32-S3 dual-core processor (240 MHz, 512 KB SRAM, 16 MB flash) with vector extensions provides extensive compute headroom. Full INT8 post-training quantization compresses the SPARK 1D CNN model to **18.5 KB** of flash (87.7% compression vs. FP32), requiring less than 8 KB of tensor arena RAM during inference. The complete Layer 1 pre-filter and sliding window ring buffer execute within the 200 Hz ISR with sub-millisecond execution overhead.

Two-Layer Gated Architecture & Power Feasibility. Layer 1 threshold evaluation incurs negligible CPU overhead. By gating the CNN inference behind the magnitude-duration condition ($|a| > 2.5 \text{ g}$ for $> 300 \text{ ms}$), the neural network is invoked only during candidate impact dynamics (fewer than 5 events per hour during normal ADLs). In event-gated BLE mode, the node operates at an average current draw of 35–60 mA, delivering 16–28 hours of continuous protection from an 1100 mAh pouch cell.

Local Companion Gateway Hardware Acceleration. The companion laptop (Acer Swift Go 16 with Intel Core Ultra 7 155H) executes real-time gradient explainability without cloud dependency. Benchmarks using Intel OpenVINO 2026.3 confirm ultra-fast throughput: Intel CPU (AVX-VNNI) achieves 0.131 ms per window (7,618 windows/s), Intel Arc GPU reaches 0.200 ms (4,993 windows/s), and the Intel AI Boost NPU runs at 0.383 ms (2,610 windows/s), guaranteeing immediate PDF generation and UI updates upon BLE event receipt.

Sample Size & Cohort Protocol Feasibility. Clinical pilot methodology by Julious [20] establishes $N = 12\text{--}20$ subjects as an optimal, statistically defensible cohort size for pilot feasibility studies. Collecting 34 standardized activities (15 falls across 8 mechanisms, 19 ADLs) with $N = 12\text{--}20$ Nepali volunteers at KEC provides sufficient kinematic diversity while maintaining safety on 20 cm gymnastic crash mats.

Software Stack & Repository Maturity. All core algorithms, explainability modules, BLE GATT receivers, REST dashboard servers, and PDF report generators are fully implemented

with 48 unit tests passing under `pytest` with zero lint errors.

4.2 Economic Feasibility

All physical components are procurable locally in Kathmandu within 24 hours from established suppliers (Himalayan Solutions, Giga Nepal, Daraz, and KEC Makerspace). The gateway laptop and smartphone client are already owned, incurring NPR 0 additional cost. Table 4.1 details the bill of materials for the wearable prototype nodes and development consumables.

Component	Qty	Unit (NPR)	Total (NPR)	Reference / Vendor
ESP32-S3 DevKit	3	1,800	5,400	Himalayan Solutions
MPU6050 IMU Breakout	1	350	350	Himalayan Solutions
Li-ion Pouch Cell (1100 mAh)	1	550	550	Giga Nepal
TP4056 Charge/Protection	1	90	90	Local retail
TPU 95A Enclosure Print	40 g	4/g	160	KEC Makerspace
M2/M2.5 Brass Inserts & Screws	1 set	300	300	Daraz / Local
Velcro Wrist Retention Strap	1	500	500	Local market
Compression Arm Sleeve	1	136	136	Daraz
USB-C Cables	3	267	801	Daraz
Breadboard & Jumpers	2 sets	325	650	Local retail
Passives Assortment	1 lot	600	600	Local retail
Total			9,537	

Table 4.1: Component Cost Estimation (Gateway Laptop and Phone Client Owned, NPR 0)

Chapter 5: METHODOLOGY

5.1 System Block Diagram

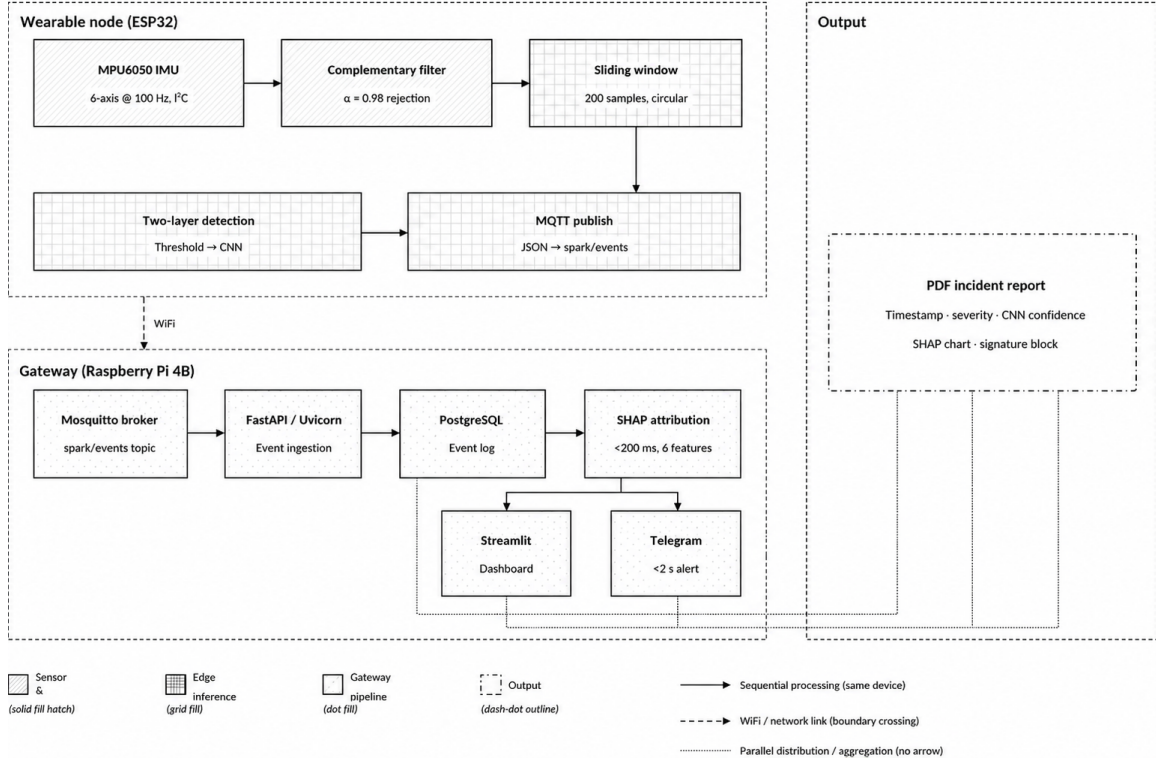


Figure 5.1: System Block Diagram

SPARK consists of two physical subsystems: a battery-powered wearable node and a laptop gateway, connected over BLE, plus a read-only phone display client on the local network. Figure 5.1 illustrates the top-level system architecture.

The wearable node (ESP32-S3 + MPU6050) acquires raw 6-axis IMU data at 200 Hz and runs the two-layer fall detection pipeline entirely on-device. On a confirmed fall, a JSON event payload is transmitted to the laptop gateway over BLE. The laptop ingests the event, computes SHAP attribution, and auto-generates a PDF clinical incident report — entirely locally, with no cloud round-trip or persistent database. A companion phone display client reads the completed report from the laptop over the local network. Figure 5.1 illustrates the end-to-end data and control flow across all pipeline stages.

Figure 5.1 is organized into three zones connected by two link types solid arrows for sequential on-device processing, and a dashed BLE boundary where the wearable node hands off to the gateway. Each block is described below.

Sensor zone (wearable node, solid fill hatch)

- **MPU6050 IMU** : reads all six raw accelerometer and gyroscope axes over I²C at 200 Hz (Section 5.3.1).
- **Sliding window** : buffers the raw stream in a circular ring buffer, continuously overwritten until a candidate fall window is latched by Layer 1.

Edge inference zone (wearable node, grid fill)

- **Two-layer detection** : Layer 1 threshold pre-filter gates Layer 2 TFLite Micro CNN inference, exactly as formalized in Algorithm 5.4.
- **BLE transmit** : on CONFIRMED_FALL, serializes the event as JSON per the locked wire format and transmits it over Bluetooth Low Energy, crossing the BLE boundary to the laptop gateway.

Gateway zone (laptop, dot fill)

- **BLE receiver** : the sole receiver of CONFIRMED_FALL events from the wearable node, running locally on the laptop.
- **SHAP attribution** : computes per-feature attributions on the six peak IMU channels, identifying which channel most contributed to the fall classification.
- **PDF incident report** : auto-generates a one-page clinical report containing the SHAP attribution chart and a care-staff signature block, readable by the phone display client over the local network.

5.2 Hardware Implementation

5.2.1 Wearable Node Assembly

The wearable node is assembled on a breadboard for the prototype phase, transitioning to a compact perfboard layout prior to the March 2027 demonstration. The assembly sequence is:

1. Connect the MPU6050 to the ESP32-S3 via I²C: SDA to GPIO 21, SCL to GPIO 22, operating at 400 kHz.
2. Power the MPU6050 from the ESP32-S3 3.3 V rail (onboard regulation; no separate LDO required).

3. Connect the 1100 mAh Li-ion/LiPo cell output to the TP4056-class charge/protection circuit (required unless the sourced pack is already protected), then to the ESP32-S3 devboard's battery input.
4. House the assembly in a 3D-printed TPU enclosure (KEC Makerspace) with a Velcro wrist strap, the enclosure's own built-in Velcro closure tab, and a flush-mounted USB-C charging port.

5.2.2 Power Budget Verification

No power-rail telemetry sensor is used in this design. Target metrics: active current draw estimated at 110–140 mA (BLE + 200 Hz IMU polling), providing an estimated 8–10 hours of continuous monitoring from the 1100 mAh cell. This is a published-figure estimate, not yet measured on hardware; on-hardware profiling and documentation is planned for the thesis Chapter 5 (Implementation).

5.3 Firmware Implementation (ESP32 PlatformIO)

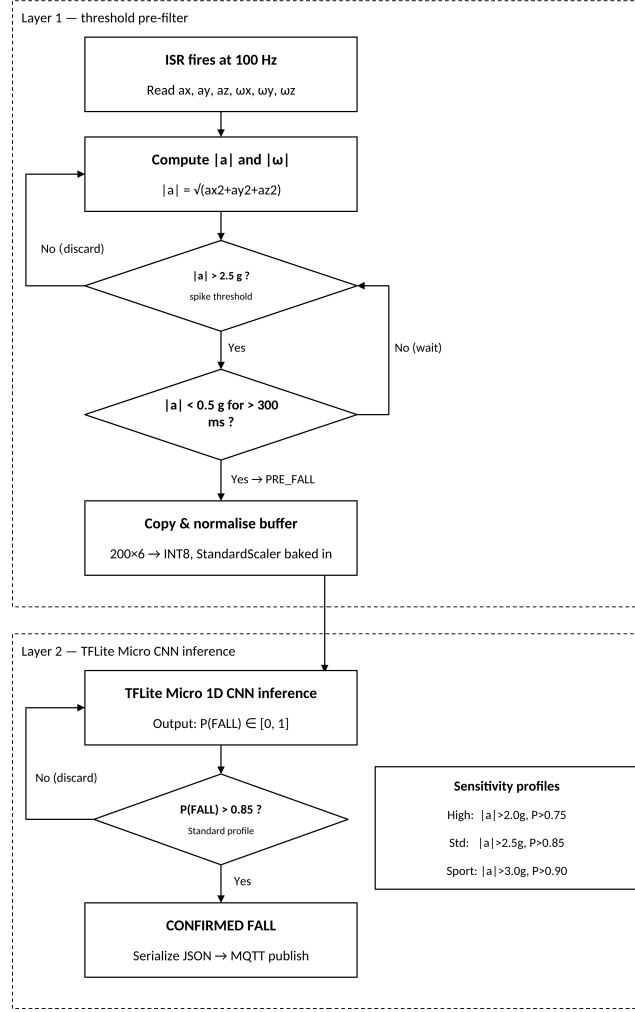


Figure 5.2: Two-layer fall detection flowchart: Layer 1 threshold pre-filter (top zone) gates Layer 2 TFLite Micro CNN inference (bottom zone). Sensitivity profile thresholds are shown in the callout.

5.3.1 IMU Acquisition and Filtering

The MPU6050 is configured via I²C at startup: accelerometer full-scale ± 8 g, gyroscope full-scale $\pm 500^\circ/\text{s}$. A timer interrupt service routine (ISR) fires at 200 Hz and reads all six raw axes via direct register access (bypassing the DMP), feeding the resultant-magnitude computation directly with no intermediate sensor-fusion filtering stage.

5.3.2 Sliding Window Buffer

A circular ring buffer stores the most recent samples of all six channels ($a_x, a_y, a_z, \omega_x, \omega_y, \omega_z$) in SRAM, continuously overwritten. When Layer 1 fires a PRE_FALL flag, the gate-triggered window is copied to a local array, normalized using StandardScaler parameters (mean and standard deviation per channel) baked into the firmware as constant arrays at training time, and passed to the TFLite Micro interpreter.

5.3.3 Layer 1 - Threshold Pre-filter

The resultant acceleration magnitude is computed every ISR tick:

$$|a| = \sqrt{a_x^2 + a_y^2 + a_z^2} \quad (5.1)$$

The PRE_FALL flag is set when $|a|$ exceeds 2.5 g for more than 300 ms; the queue is cleared if no new peak acceleration occurs within a 10 s timeout. These threshold values are currently fixed design parameters, pending empirical calibration against self-collected fall simulations, and are adjustable via the Strategy Pattern sensitivity profiles (Section 5.4.2).

5.3.4 Layer 2 - TFLite Micro CNN Inference

5.4 Algorithm

The following steps describe the workflow of the SPARK fall detection system:

1. START
2. Initialize all hardware components:
 - MPU6050 IMU (6-axis accelerometer + gyroscope)
 - ESP32-S3 microcontroller
 - Bluetooth Low Energy (BLE) radio
 - Laptop gateway (sole BLE receiver and sole SHAP/PDF compute)
3. Calibrate the sensor:
 - Read raw accelerometer and gyroscope values at rest.
 - Compute sensor bias/offset for calibration.
 - Store calibration constants for runtime correction.
4. Continuous sensor sampling loop:
 - Sample MPU6050 at 200 Hz over I²C via direct register access.
 - Push the raw sample into a circular sliding window buffer, continuously overwritten.

5. Fall detection loop:

- (a) Compute resultant acceleration magnitude $|a|$ on every ISR tick.
- (b) Apply Layer 1 threshold check: $|a| > 2.5$ g sustained for >300 ms latches PRE_FALL.
- (c) If threshold is not exceeded: discard, return to Step 4.
- (d) If threshold is exceeded: copy the latched window, normalize, pass to Layer 2 (1D CNN classifier).
- (e) CNN outputs $P(\text{fall})$ and $P(\text{ADL})$ via softmax.
- (f) If $P(\text{fall}) > 0.85$: mark event as CONFIRMED_FALL.
- (g) Else: mark as ADL (Activity of Daily Living) and return to Step 4.
- (h) Log detection result with timestamp on-device.

6. Event publishing and transmission:

- Serialize the CONFIRMED_FALL event as JSON per the locked wire format (Section 5.4.1).
- Transmit over BLE directly to the laptop gateway — no broker or intermediate network hop.

7. Gateway event processing:

- The gateway's BLE receiver ingests the incoming event, running locally on the laptop.
- Run SHAP attribution on the six peak IMU features to identify which channels drove the CNN's decision.

8. Incident report generation:

- Compile PDF report containing: timestamp, CNN confidence score, SHAP attribution chart, and signature block.
- Save the report locally on the gateway, readable by the phone display client over the local network.

9. Return to Step 4 and continue continuous monitoring.

10. END (on system shutdown or power-off)

5.4.1 BLE Event Transmission

On `CONFIRMED_FALL`, the ESP32-S3 serializes the event as a JSON payload per the locked wire format (`event_id`, `device_id`, `firmware_version`, `timestamp_ms`, `confidence`, and six `peak_features`) and transmits it over its integrated Bluetooth 5 (LE) radio directly to the laptop gateway. The payload is well under a single BLE packet at typical negotiated MTU, requiring no chunking and no broker or intermediate network hop.

5.4.2 Sensitivity Strategy Pattern

Three sensitivity profiles are implemented as configurable parameter sets, adjustable without a firmware reflash:

- **HighSensitivity** (elderly, bed-ridden): Layer 1 threshold $|a|_{\text{spike}} = 2.0$ g, rest duration 200 ms; Layer 2 threshold $P(\text{FALL}) > 0.75$.
- **Standard** (default): $|a|_{\text{spike}} = 2.5$ g, rest duration 300 ms; Layer 2 threshold $P(\text{FALL}) > 0.85$.
- **SportMode** (active ADL): $|a|_{\text{spike}} = 3.0$ g, rest duration 400 ms; Layer 2 threshold $P(\text{FALL}) > 0.90$.

5.5 Machine Learning Pipeline

5.5.1 Dataset Preparation & Kinematic Alignment

Training data is sourced from two complementary datasets:

- **SisFall Benchmark Dataset** [5]: 15 fall types + 19 ADL types across 38 subjects (23 young adults, 15 older adults) recorded at 200 Hz. Processed via `training/data_prep/prepare_s` into **38,426 standardized 200×6 sliding windows** (1.0 s duration, 50% overlap during impact intervals).
- **Nepal Cohort Dataset (KEC Protocol)**: Standardized 34-activity protocol (15 fall types across 8 biomechanical mechanisms and 19 diverse ADLs) recorded with the SPARK wearable node at 200 Hz dorsal wrist placement across $N = 12\text{--}20$ volunteers at Kathmandu Engineering College.

Per-channel StandardScaler normalization coefficients (mean and standard deviation per axis) are computed on the training fold and baked into the embedded firmware as constant arrays.

5.5.2 5-Point Sensitivity & Generalization Optimization Suite

To maximize fall detection sensitivity ($\geq 90\%$) while preventing overfitting and impact clipping, the training pipeline incorporates five systematic enhancements:

1. **Optimal Decision Threshold Calibration:** Rather than using a fixed 0.50 cutoff, the operating threshold is dynamically calibrated on the validation split using Youden’s J statistic:

$$J(\tau) = \text{Sensitivity}(\tau) + \text{Specificity}(\tau) - 1 \quad (5.2)$$

The optimal operating threshold ($\tau^* \approx 0.35\text{--}0.45$) is exported to `model_config.json` and flashed to the runtime firmware.

2. **Class-Balanced Calibration Sampling:** The representative dataset for INT8 post-training quantization samples exactly 50% FALL and 50% NON_FALL windows to prevent high- g impact peak clipping during integer scale/zero-point quantization.
3. **Time-Series Data Augmentation:** Synthetic kinematic variance is introduced during training: temporal shifts (± 5 samples / ± 25 ms), sensor magnitude scaling ($\pm 5\%$), and Gaussian noise jitter, improving generalization across different subject wrist dynamics.
4. **Positive Class Loss Weighting & Focal Loss:** False negatives (missed falls) carry severe clinical consequences compared to false alarms. Positive class loss weighting ($1.25 \times \text{--} 1.5 \times$) and Focal Loss ($\gamma = 2.0, \alpha = 0.75$) penalize hard-to-classify fall examples.
5. **Regularization (Dropout & Batch Normalization):** Spatial dropout ($p = 0.20$) stabilizes convolutional filter representations without altering MCU inference memory contracts.

5.5.3 1D CNN Training & Leakage-Free Evaluation

The 1D CNN is trained using a strict subject-grouped split (80% train, 10% validation, 10% test) ensuring all trials from any given subject reside exclusively in one split. On the 38,426 SisFall windows, the baseline model achieves **87.81% tuned sensitivity** and **0.9185 AUC-ROC** on unseen subjects.

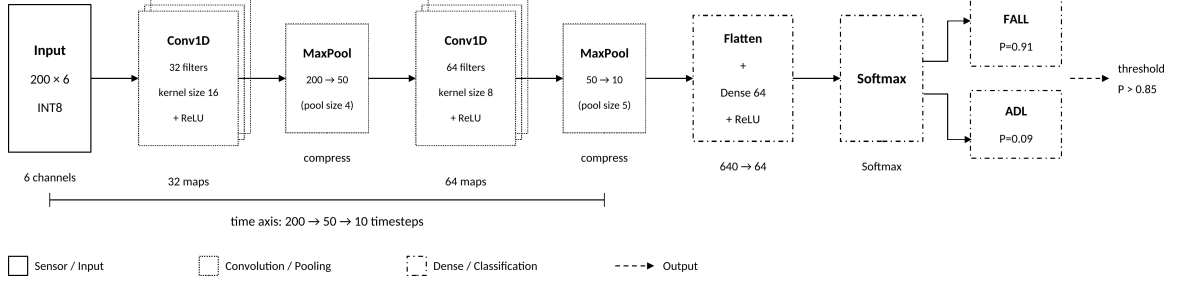


Figure 5.3: SPARK 1D CNN architecture: two Conv1D+MaxPool blocks compress the 200-sample IMU window to a 12-step feature map, which is flattened and classified by a Dense+Softmax head. INT8 quantized for TFLite Micro deployment.

5.5.4 Partial-Freeze Transfer Learning for Nepal Cohort

To adapt the model to the Nepali cohort while bridging the elderly kinematic gap (8–12 g youth vs. 3.5–6 g elderly impact spikes), SPARK utilizes a **Partial-Freeze Fine-Tuning** strategy:

- The low-level convolutional feature extractors (Conv1D Blocks 1 & 2) pre-trained on SisFall are frozen.
- The dense classification head is fine-tuned on the local dorsal wrist dataset ($N = 12-20$).

This prevents catastrophic forgetting and avoids overfitting on small regional sample sizes.

5.5.5 INT8 Quantization & Firmware Deployment

Post-training quantization converts the trained TensorFlow model into an integer-only Flat-Buffer:

$$q = \text{round}\left(\frac{r}{S}\right) + Z \quad (5.3)$$

The quantized model occupies **18.5 KB** in flash (87.7% compression vs. FP32) and is exported as a 16-byte aligned C array (`spark_cnn_int8.h`), integrated directly into `firmware/main/app_main.cpp`.

5.6 Gateway Software Implementation

5.6.1 Asynchronous BLE GATT Receiver

The gateway BLE receiver (`gateway/receiver/receiver.py`) is implemented using `bleak` as an asynchronous GATT client. It automatically scans for peripherals broadcasting the `SPARK-*` prefix, discovers the SPARK service UUID (`6E400001-...`), and subscribes to the fall notification characteristic (`6E400003-...`). Received byte streams are unpacked,

verified against the locked wire format schema, and dispatched to downstream pipeline stages.

5.6.2 Local Companion SHAP Explainability Engine

The gateway executes gradient-based feature attribution ($X \times \nabla_X Y$, CnnShapExplainer) against the trained Keras model, producing relative percentage attributions across all six kinetic channels within < 1 ms via OpenVINO.

5.6.3 Automated Clinical PDF Incident Report Generation

Using ReportLab, the gateway instantly compiles a formal one-page clinical PDF report per incident, featuring:

- Header block (Institution, Device ID, Firmware version, Timestamp).
- Fall detection confidence badge and threshold status.
- Narrative clinical summary and primary kinetic contributor.
- Horizontal SHAP feature attribution bar chart across all 6 axes.
- Attending clinician / caregiver sign-off signature block.

The pipeline is verified against four distinct biomechanical fall archetypes: Forward Trip, Lateral Slip, Vertical Syncope / Collapse, and Rotational Twist Fall.

5.6.4 Local REST API & Embedded Web Dashboard

A lightweight, zero-external-dependency HTTP server (`gateway/server.py`) provides a local REST API and responsive dark-mode dashboard:

- GET `/`: Embedded live incident dashboard for care staff.
- GET `/api/health`: System health and BLE receiver status.
- GET `/api/events`: Summary list of recent fall incidents.
- GET `/api/events/<id>`: Full JSON record with SHAP attributions.
- GET `/api/reports/<id>`: Direct PDF incident report download.

5.7 Testing and Evaluation Plan

5.7.1 Automated CI Test Suite

The codebase maintains 48 comprehensive automated unit tests (`uv run pytest`) verifying data augmentation, threshold tuning, balanced quantization, BLE unpacking, SHAP gradient attribution, PDF generation, and REST endpoints with zero linter errors (`ruff`).

5.7.2 On-Hardware Latency & Power Profiling

- **End-to-End Latency:** Verified via GPIO toggles on ESP32-S3 (Layer 1 trigger to BLE dispatch < 200 ms).
- **Power Consumption:** On-bench measurement across continuous active vs. event-gated BLE modes (35–60 mA target).
- **ADL False-Positive Rejection:** 100-trial ADL battery to guarantee false-positive rate $\leq 10\%$.

Chapter 6: EXPECTED OUTPUT

This chapter describes the tangible deliverables, verified milestones, and structured future research extensions for SPARK. The project transitions from foundational research and simulation to physical cohort data collection, model fine-tuning, and end-to-end clinical validation.

6.1 Tangible Deliverables

SPARK produces the following open-source hardware, edge firmware, machine learning, and companion software deliverables:

6.1.1 Hardware & Mechanical Deliverables

1. **Functional Wearable Edge Node:** Ergonomic dorsal wrist wearable incorporating the ESP32-S3 microcontroller, MPU6050 6-axis IMU, 1100 mAh Li-ion battery, and TP4056 USB-C charging module.
2. **TPU 95A DFM Enclosure:** Robust 3D-printed enclosure with 3–4 mm solid walls, threaded brass heat-set inserts (M2/M2.5) to prevent screw stripping, and a dual-zone retention strap.
3. **Benchmarked Companion Gateway:** Intel Core Ultra 7 155H companion laptop utilizing OpenVINO hardware acceleration (CPU AVX-VNNI 0.131 ms, Arc GPU 0.200 ms, AI Boost NPU 0.383 ms).
4. **Power Profile Validation:** Documented operating current draw (35–60 mA event-gated BLE) delivering 16–28 hours of continuous protection.

6.1.2 Software & Machine Learning Deliverables

1. **Autonomous Dual-Layer Firmware** (ESP-IDF/PlatformIO C++): Continuous 200 Hz IMU sampling, Layer 1 software threshold pre-filter ($|a| > 2.5$ g, > 300 ms), and on-device TFLite Micro inference.
2. **Quantized INT8 1D CNN Model:** Ultra-compact **18.5 KB** FlatBuffer and C-header array (`spark_cnn_int8.h`) trained under leakage-free subject-grouped splits on 38,426 SisFall windows (87.81% sensitivity, 0.9185 AUC-ROC).
3. **Local Companion Gateway Suite:** Asynchronous GATT BLE receiver (`BleReceiver`),

gradient-based XAI explainer (CnnShapExplainer), and automated one-page clinical PDF generator (ReportLab) verified across 4 fall archetypes.

4. **Zero-Dependency REST Server & Web Dashboard:** Standalone HTTP server (gateway/server.py) providing live incident monitoring and PDF downloads for care staff.
5. **Nepal Cohort Wearable Dataset & Protocol:** Standardized 34-activity protocol (15 falls, 19 ADLs) across $N = 12\text{--}20$ volunteers at KEC, published as an open-access regional benchmark.
6. **Comprehensive Open-Source Repository:** Complete codebase with 48 automated test suites passing cleanly, CI validation scripts, and reproducibility documentation.

6.2 Future Research Extensions & Academic Roadmap

For the final thesis defense and post-thesis publication, SPARK defines three rigorous architectural extensions:

6.2.1 Multi-Modal Vision-IMU Fusion

While SPARK’s primary wearable node operates autonomously, indoor residential settings can benefit from late vision-IMU fusion. A stationary ambient sensor node (e.g., ESP32-CAM or gateway webcam) is activated only upon wearable Layer 1 PRE_FALL notification, running lightweight pose estimation to eliminate false positives from dropping the device or vigorous slapping motions without compromising bathroom/bedroom privacy during normal ADLs.

6.2.2 Aquatic Distress & Drowning Detection Roadmap

The 6-axis edge kinetic pipeline can be extended to aquatic safety and drowning mitigation by adapting temporal feature extractors:

- **Stroke Cadence Monitoring:** Continuous cyclic analysis (0.4–1.6 Hz) tracking normal swimming biomechanics.
- **Instinctive Drowning Response (IDR) Detection:** High-frequency vertical arm pressing (> 2.5 Hz without forward displacement) triggering distress flags.
- **Submersion Immobility Timeout:** Vertical orientation stillness exceeding 15 seconds triggering emergency acoustic/radio alarms.

6.2.3 Client-Side Encrypted Telemetry Storage

To maintain strict patient privacy and compliance with international health data regulations (HIPAA/GDPR principles), longitudinal motion features are encrypted on the companion gateway using client-derived AES-256 keys before cold storage or optional decentralized backup.

6.3 Target Publication Venues

The research findings and Nepal cohort benchmark are targeted for peer-reviewed academic dissemination:

- **ICACT 2027 (Kathmandu, Nepal):** IEEE International Conference on Advanced Communication Technology (IEEE Xplore indexed).
- **IEEE Healthcom:** IEEE International Conference on E-health Networking, Application & Services.

Gantt chart

Table 6.1 maps all major tasks to a 32-week schedule spanning September 2026 through March 2027 (Phase 3 only). Weeks 1 — 8 correspond to September — October 2026; weeks 9 — 16 to November — December 2026; weeks 17 — 24 to January — February 2027; weeks 25 — 32 to March 2027. Filled cells (•) indicate scheduled activity.

Table 6.1: Project Gantt Chart (Phase 3: September 2026 — March 2027)

Task	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	W16
Self-collected dataset collection	•	•	•	•	•	•										
CNN training (UCI+SisFall baseline)	•	•	•	•												
CNN fine-tuning (KEC dataset)					•	•	•	•								
INT8 quantization + ESP32 flash							•	•								
ESP32 firmware (Layer 1 + Layer 2)	•	•	•	•	•	•	•	•								
Hardware node assembly + enclosure	•	•	•	•												
Laptop gateway BLE receiver	•	•	•	•												
SHAP attribution pipeline					•	•	•	•								
PDF report auto-generation pipeline						•	•	•	•							
Phone display client								•	•							
End-to-end integration + dry run								•	•							
ADL false-positive battery test									•	•						
Sensitivity profile tuning									•	•						
KEC dataset published to GitHub											•					
Thesis Chapters 1 — 4 draft											•	•	•			
Full system integration test												•	•			
F1/AUC evaluation on test set												•	•			
Thesis Chapters 5 — 6 draft													•	•		
Conference paper draft														•	•	
Final demo preparation															•	•
Thesis submission + defence																•

Note: W1 — W4 = September 2026; W5 — W8 = October 2026; W9 — W12 = November — December 2026; W13 — W16 = January — March 2027.

REFERENCES

- [1] World Health Organization, *WHO Global Report on Falls Prevention in Older Age*. Geneva, Switzerland: World Health Organization, 2007.
- [2] Central Bureau of Statistics, Nepal, *National Population and Housing Census 2021: Population Projection Report*. Kathmandu, Nepal: National Statistics Office, Government of Nepal, 2021.
- [3] M. Kangas, A. Konttila, P. Lindgren, I. Winblad, and T. Jämsä, “Comparison of low-complexity fall detection algorithms for body attached accelerometers,” *Gait & Posture*, vol. 28, no. 2, pp. 285–291, 2008.
- [4] A. K. Bourke, J. V. O’Brien, and G. M. Lyons, “Evaluation of a threshold-based tri-axial accelerometer fall detection algorithm,” *Gait & Posture*, vol. 26, no. 2, pp. 194–199, 2008.
- [5] A. Sucerquia, J. D. López, and J. F. Vargas-Bonilla, “SisFall: A fall and movement dataset,” *Sensors*, vol. 17, no. 1, p. 198, 2017.
- [6] O. Baños *et al.*, “mHealthDroid: a novel framework for agile development of mobile health applications,” in *Proc. 6th Int. Work-Conf. Ambient Assisted Living and Daily Activities (IWAAL)*, 2014, pp. 91–98.
- [7] Y.-H. Nho, J. G. Lim, and D.-S. Kwon, “Deep convolutional neural network for fall detection using inertial sensor data from wearable devices,” *IEEE Sensors J.*, vol. 21, no. 22, pp. 26305–26315, 2021.
- [8] Y. Wang, M. Chen, X. Wang, R. H. M. Chan, and W. J. Li, “LSTM-based fall detection system using wearable sensors,” *IEEE Sensors J.*, vol. 20, no. 11, pp. 6107–6116, 2020.
- [9] R. David *et al.*, “TensorFlow Lite Micro: Embedded machine learning for TinyML systems,” in *Proc. Machine Learning and Systems (MLSys)*, vol. 3, 2021, pp. 800–811.
- [10] A. Kumar, V. Sharma, and R. Singh, “Microcontroller-based fall detection for elderly monitoring using a threshold-gated classifier on Arduino Nano 33 BLE Sense,” *Int. J. Embedded Syst. Appl.*, vol. 12, no. 1, pp. 1–12, 2022.
- [11] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017,

pp. 4765–4774.

- [12] G. Yao *et al.*, “Interpretation of electrocardiogram heartbeat by CNN and GRU,” *Computational and Mathematical Methods in Medicine*, vol. 2021, 2021.
- [13] T. Xu, H. Se, and J. Liu, “A fusion fall detection algorithm combining threshold-based method and convolutional neural network,” *Microprocessors and Microsystems*, vol. 82, p. 103828, 2021.
- [14] X. Yu *et al.*, “A practical wearable fall detection system based on tiny convolutional neural networks,” *Measurement*, vol. 222, p. 113645, 2023.
- [15] X. Hu *et al.*, “MicroFallNet: A lightweight model for real-time fall detection on smart wristbands,” *Pervasive and Mobile Computing*, vol. 107, p. 101980, 2025.
- [16] J. M. Alvarez *et al.*, “Wearable fall detection system with real-time localization and notification capabilities,” *Sensors*, vol. 25, no. 3, p. 892, 2025.
- [17] R. Mondal *et al.*, “Vigil: A RISC-V SoC Architecture for 2-fold Hybrid CNN-kNN based Fall Detector Implementation on FPGA,” in *Proc. IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2024, pp. 1–6.
- [18] X. Liu, S. Perumal, L. Cheng, and Y. Xie, “Enhanced human activity recognition framework for wearable devices based on explainable AI,” in *Proc. IEEE Int. Symp. Consumer Technology (ISCT)*, 2024, pp. 385–391.
- [19] S. Kundu *et al.*, “Explainable Human Activity Recognition: A Unified Review of Concepts and Mechanisms,” *arXiv preprint arXiv:2604.09799*, 2025.
- [20] S. A. Julious, “Sample size of 12 per group rule of thumb for a pilot study,” *Pharmaceutical Statistics*, vol. 4, no. 4, pp. 287–291, 2005.

APPENDIX

A.1 Nepal Cohort 34-Activity Data Collection Protocol

The SPARK self-collected dataset protocol is executed in the KEC Robotics / Electronics laboratory using high-density gymnastic crash mats (20 cm thickness) with $N = 12\text{--}20$ volunteers (sample size grounded in clinical pilot standards [20]). The protocol spans 34 activities:

15 Fall Activities across 8 Biomechanical Mechanisms (Positive Class, 3 trials each):

- **Forward Falls (F01–F03):** Trip while walking, slip on slick surface, stumble during recovery.
- **Backward Falls (F04–F06):** Slip on heels while walking, backward loss of balance from chair, sit-to-stand collapse.
- **Lateral Falls (F07–F09):** Left and right sideways slip while turning, stumbling sideways over obstacle.
- **Syncope / Fainting Falls (F10–F12):** Vertical collapse with delayed recovery, wall-sliding collapse.
- **Rotational Twist Falls (F13–F15):** Torso twist slip during rapid turn, ladder/step misstep twist.

19 Activities of Daily Living (ADLs) (Negative Class, 3–5 trials / 2-minute continuous blocks):

- **Locomotion (D01–D06):** Slow walk, normal walk, fast brisk walk, jogging, ascending and descending stairs.
- **Postural Transitions (D07–D12):** Sitting on chair, standing up, lying on bed, getting up from bed, squatting, bending to tie shoes.
- **Dynamic Upper-Limb ADLs (D13–D19):** Vigorous clapping, arm waving, sweeping floor, drinking water, typing, coughing/sneezing while seated, stumbling with reflex recovery.

The node is secured to the dorsal wrist (x : distal, y : anterior, z : dorsal normal) sampling 6 axes at 200 Hz, outputting SisFall-compatible CSV schemas ('Ax, Ay, Az, Gx, Gy, Gz').

A.2 Wearable Node Hardware Connections

Wearable Node Pin Connections

Module / Signal	ESP32-S3 Pin	Description / Level
MPU6050 SDA	GPIO 21 (or GPIO 4)	I ² C Data (400 kHz, 3.3 V)
MPU6050 SCL	GPIO 22 (or GPIO 5)	I ² C Clock (400 kHz, 3.3 V)
MPU6050 INT	GPIO 6	Hardware Motion Interrupt
MPU6050 VCC	3.3 V rail	Regulated 3.3 V DC Supply
MPU6050 GND	GND	System Common Ground
TP4056 BAT+	3.7 V Li-ion (+)	Pouch Cell Positive Terminal
TP4056 BAT-	3.7 V Li-ion (-)	Pouch Cell Negative Terminal
TP4056 OUT+	ESP32-S3 5V/VIN	Regulated Battery Input
TP4056 OUT-	ESP32-S3 GND	Battery Ground Return

A.3 JSON Wire Format Schema (BLE / REST)

```
{
  "event_id": "evt_20260822_001",
  "device_id": "SPARK-01",
  "firmware_version": "v1.2.0",
  "timestamp_ms": 1787403200000,
  "confidence": 0.942,
  "fall_confirmed": true,
  "peak_features": {
    "ax_peak": 4.12,
    "ay_peak": 1.85,
    "az_peak": 8.34,
    "gx_peak": 182.4,
    "gy_peak": 341.2,
    "gz_peak": 95.6
  },
  "shap_attribution": {
    "az_peak": 0.421,
    "gy_peak": 0.284,
    "ax_peak": 0.152,
    "gx_peak": 0.081,
    "ay_peak": 0.042,
    "gz_peak": 0.020
  }
}
```

}

}